organized. This makes it easier to locate and maintain code specific to a feature or functionality.

- *Clean URLs*—By using a route group, the folder name (chat) is excluded from the URL path. This results in a cleaner and more user-friendly URL for the application without having to configure anything.
- *Independent layouts*—If you plan to have a distinct layout for the chat section compared to other parts of your application, using a route group allows you to define a separate layout.js file within a group. This ensures that certain pages inherit the specific layout defined within the group.

Interestingly, there's no page.js file directly inside the app folder. So how does Next.js know where to find the index page component? This is because the name you chose for the route (chat) has no effect on the URL path itself. It's purely for organizational purposes within the project structure. In essence, Next.js actively searches for a page.js file within the (chat) folder and uses it as the main component for the chat page, which is also the index page component.

2.8.5 Layout components

In the Next.js project codebase, we renamed the layout component (AppLayout.jsx) to layout.js and moved it inside the (chat) route group. Why is that? This is another file convention that Next.js supports, which allows us to create a hierarchy of layouts. By placing a layout.js file within a folder in the app directory, you define a layout specific to that route segment. These layouts can then be nested within one another.

As an example of how this works, imagine a website with a global header and navigation bar present on all pages. Additionally, you might have a section dedicated to blog posts, with its own specific layout elements, such as sidebars or featured content areas. Here's how nesting layouts can achieve this:

- *Root layout*—A root layout, typically placed at app/layout.js, defines the overall application structure, including the global header and navigation. This layout would wrap all other layouts and pages within your application.
- *Nested layout for blog section*—Within a folder named blog (representing the blog section), you can create a layout.js file. This layout would define elements specific to the blog section, such as sidebars or featured content areas. It would inherit the global elements from the root layout.
- *Individual blog posts*—Each blog post would likely be a separate page (page.js) within the blog route segment (under a folder wrapped in brackets like [post]). These pages would inherit the layouts from both the root and the blog section layout, resulting in a consistent experience with global elements and blog-specific elements.

Figure 2.10 demonstrates visually how you should structure your folders to accommodate different layout trees for each use case.